

1. First Palindromic String in an Array

main.py	Visualize	Run ▶	Output
<pre>1 words=["abc","car","ada","racecar","cool"] 2 for word in words: 3 if word ==word[::-1]: 4 print(word) 5 break 6 else: 7 print(" ")</pre>			ada

2. Count Common Elements in Two Arrays

main.py	Visualize	Run ▶	Output
<pre>1 nums1 = [4, 3, 2, 3, 1] 2 nums2 = [2, 2, 5, 2, 3, 6] 3 answer1 = 0 4 answer2 = 0 5 for num in nums1: 6 if num in nums2: 7 answer1 += 1 8 for num in nums2: 9 if num in nums1: 10 answer2 += 1 11 print([answer1, answer2])</pre>			[3, 4]

3. Sum of Squares of Distinct Counts

main.py	Visualize	Run ▶	Output
<pre>1 nums = [1, 2, 1] 2 answer = 0 3 for i in range(len(nums)): 4 distinct = set() 5 for j in range(i, len(nums)): 6 distinct.add(nums[j]) 7 count = len(distinct) 8 answer += count * count 9 print(answer)</pre>			15

4. Count Equal Pairs Divisible by K

main.py	Visualize	Run ▶	Output
<pre>1 nums = [3, 1, 2, 2, 2, 1, 3] 2 k = 2 3 count = 0 4 for i in range(len(nums)): 5 for j in range(i + 1, len(nums)): 6 if nums[i] == nums[j] and (i * j) % k == 0: 7 count += 1 8 print(count)</pre>			4

5. Find the Maximum Element in an Array

main.py	Visualize	Run ▶	Output
<pre>1 nums = [1, 2, 3, 4, 5] 2 maximum = nums[0] 3 for num in nums: 4 if num > maximum: 5 maximum = num 6 print(maximum)</pre>			5

6. Sort the List and Find Maximum Element

main.py	Visualize	Run ▶	Output
<pre>1 def find_max(nums): 2 if not nums: 3 return None 4 nums.sort() 5 return nums[-1] 6 print(find_max([])) 7 print(find_max([5])) 8 print(find_max([3, 3, 3, 3, 3]))</pre>			None 5 3

7. Remove Duplicate Elements from a List

main.py	Visualize	Run ▶	Output
<pre>1 def unique_elements(nums): 2 unique = [] 3 for num in nums: 4 if num not in unique: 5 unique.append(num) 6 return unique 7 print(unique_elements([3, 7, 3, 5, 2, 5, 9, 2])) 8 print(unique_elements([-1, 2, -1, 3, 2, -2])) 9 print(unique_elements([1000000, 999999, 1000000]))</pre>			<pre>[3, 7, 5, 2, 9] [-1, 2, 3, -2] [1000000, 999999]</pre>

8. Bubble Sort

main.py	Visualize	Run ▶	Output
<pre>nums = [5, 2, 8, 1, 3] n = len(nums) for i in range(n): for j in range(n - i - 1): if nums[j] > nums[j + 1]: nums[j], nums[j + 1] = nums[j + 1], nums[j] print(nums)</pre>			<pre>[1, 2, 3, 5, 8]</pre>

9. Binary Search

main.py	Visualize	Run ▶	Output
<pre>1 arr = [3, 4, 6, -9, 10, 8, 9, 30] 2 key = 10 3 arr.sort() 4 low = 0 5 high = len(arr) - 1 6 while low <= high: 7 mid = (low + high) // 2 8 if arr[mid] == key: 9 print("Element", key, "is found at position", mid + 1) 10 break 11 elif arr[mid] < key: 12 low = mid + 1 13 else: 14 high = mid - 1 15 else: 16 print("Element", key, "is not found")</pre>			<pre>Element 10 is found at position 7</pre>

10. Sort Array in $O(n \log n)$

main.py	Visualize	Run ▶	Output
<pre>1 def merge_sort(a): 2 if len(a) <= 1: 3 return a 4 mid = len(a) // 2 5 left = merge_sort(a[:mid]) 6 right = merge_sort(a[mid:]) 7 result = [] 8 while left and right: 9 if left[0] < right[0]: 10 result.append(left.pop(0)) 11 else: 12 result.append(right.pop(0)) 13 return result + left + right 14 nums = [5, 2, 8, 1, 3] 15 print(merge_sort(nums))</pre>			[1, 2, 3, 5, 8]

11. Number of Ways to Move Ball Out of Grid

main.py	Visualize	Run ▶	Output
<pre>1 def findWays(m, n, N, i, j): 2 if N == 0: 3 return 0 4 if i < 0 or i >= m or j < 0 or j >= n: 5 return 1 6 return (findWays(m, n, N-1, i+1, j) + 7 findWays(m, n, N-1, i-1, j) + 8 findWays(m, n, N-1, i, j+1) + 9 findWays(m, n, N-1, i, j-1)) 10 print(findWays(2, 2, 2, 0, 0)) 11 print(findWays(1, 3, 3, 0, 1))</pre>			2 8

12. House Robber II

main.py	Visualize	Run ▶	Output
<pre>1 def rob(nums): 2 def simple_rob(a): 3 prev = curr = 0 4 for money in a: 5 prev, curr = curr, max(curr, prev + money) 6 return curr 7 if len(nums) == 1: 8 return nums[0] 9 return max(simple_rob(nums[:-1]), simple_rob(nums[1:])) 10 nums = [2, 3, 2] 11 print(rob(nums))</pre>			3

13. Climbing Stairs

main.py	Visualize	Run ▶	Output
<pre>1 n = 4 2 a = 1 3 b = 1 4 for i in range(n): 5 a, b = b, a + b 6 print(a)</pre>			5

14. Unique Paths in a Grid

main.py	Visualize	Run ▶	Output
<pre>1 m = 7 2 n = 3 3 dp = [1] * n 4 for i in range(1, m): 5 for j in range(1, n): 6 dp[j] = dp[j] + dp[j - 1] 7 print(dp[-1])</pre>			28

15. Find Large Groups in a String

main.py	Visualize	Run ▶	Output
<pre>1 s = "abbxxxzzy" 2 result = [] 3 i = 0 4 while i < len(s): 5 j = i 6 while j < len(s) and s[j] == s[i]: 7 j += 1 8 if j - i >= 3: 9 result.append([i, j - 1]) 10 i = j 11 print(result)</pre>			[[3, 6]]

16. Game of Life

main.py	Visualize	Run ▶	Output
<pre>1 board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]] 2 m, n = len(board), len(board[0]) 3 new = [[0]*n for _ in range(m)] 4 for i in range(m): 5 for j in range(n): 6 count = sum(board[x][y] for x in range(max(0,i-1),min(m,i+2)) 7 for y in range(max(0,j-1),min(n,j+2)) 8 if (x,y) != (i,j)) 9 if count == 3 or (board[i][j] == 1 and count == 2): 10 new[i][j] = 1 11 print(new)</pre>			[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]

17. Champagne Tower

main.py		Visualize	Run ▶	Output
1	poured = 1			0.0
2	row = 1			
3	glass = 1			
4	cups = [[0] * 101 for _ in range(101)]			
5	cups[0][0] = poured			
6	for i in range(row):			
7	for j in range(i + 1):			
8	extra = max(0, cups[i][j] - 1) / 2			
9	cups[i + 1][j] += extra			
10	cups[i + 1][j + 1] += extra			
11	print(min(1, cups[row][glass]))			